



INSTITUTO POLITÉCNICO NACIONAL

---

UPIITA - Unidad Profesional  
Interdisciplinaria en Ingeniería y  
Tecnologías Avanzadas IPN

Base de Datos Distribuidas

Práctica 4 CONCURRENCIA

Marco Gonzalez Luna 2024640044

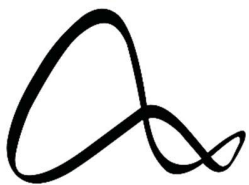
Olvera Valdivia Cristobal 2024640014

Grupo

3TM2

Profesor

Carlos De La Cruz Sosa



upiita-ipn

# 1. Dependencia No Confirmada (Lectura Sucia)

## Descripción del problema:

Este fenómeno se manifiesta cuando una transacción recupera información de la base de datos sin establecer los mecanismos de bloqueo de lectura correspondientes. En consecuencia, el sistema gestor permite que la conexión acceda a registros que están siendo modificados simultáneamente por otra transacción que aún no ha finalizado (estado uncommitted). Si la transacción originaria de los cambios ejecuta una reversión de sus operaciones (ROLLBACK), los datos utilizados por la lectura inicial resultan ser ficticios o inconsistentes. A esta vulnerabilidad se le denomina “lectura de datos sucios”, ya que el procesamiento y los cálculos subsecuentes dependen de un estado transitorio de la base de datos, lo que compromete gravemente la integridad del sistema al fundamentarse en información que nunca se consolidó de manera permanente.

## Código de ejemplo:

Conexión A:

```
USE AdventureWorks2022;

GO
BEGIN TRAN;

-- 1. Aplicación un cambio temporal
UPDATE HumanResources.Employee
SET VacationHours = 200
WHERE BusinessEntityID = 1;

-- 2. Dar tiempo para que la Conexión B lea el dato sucio
WAITFOR DELAY '00:00:10';

-- 3. Revertir el cambio.
ROLLBACK TRAN;
```

---

(1 fila afectada)

Hora de finalización: 2026-09-22T06:10:32.6090499+02:00

Conexión B:

```
USE AdventureWorks2022;
```

```
GO
```

```
-- Se fuerza a SQL Server a ignorar los bloqueos exclusivos de otras transacciones
```

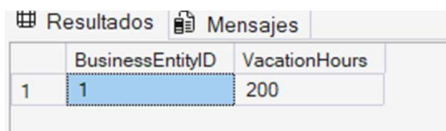
```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

```
-- Esto leerá "200" en VacationHours, un dato que será revertido en breve
```

```
SELECT BusinessEntityID, VacationHours
```

```
FROM HumanResources.Employee
```

```
WHERE BusinessEntityID = 1;
```



The screenshot shows a SQL Server query results window with two tabs: 'Resultados' (Results) and 'Mensajes' (Messages). The 'Resultados' tab is active, displaying a table with two columns: 'BusinessEntityID' and 'VacationHours'. The table contains one row with the values '1' and '200' respectively. The row is highlighted with a blue background.

|   | BusinessEntityID | VacationHours |
|---|------------------|---------------|
| 1 | 1                | 200           |

### Conclusión:

En este escenario, la Conexión B leyó un valor ('*VacationHours* = 200') que nunca se consolidó en la base de datos debido a que la Conexión A ejecutó un ROLLBACK. Si se tomaran decisiones basadas en esa lectura, el sistema presentaría fallas de integridad. SQL Server 2000 y sus versiones posteriores previenen este problema automáticamente estableciendo READ COMMITTED como el nivel de aislamiento predeterminado, el cual obliga a que las conexiones esperen a que las modificaciones se confirmen antes de poder leerlas.

## 2. Análisis Inconsistente (Lectura No Repetible)

### Descripción del problema:

Este problema de concurrencia se manifiesta en entornos relacionales cuando una transacción de larga duración recupera un mismo conjunto de datos en múltiples ocasiones consecutivas. Si la configuración de aislamiento transaccional carece de bloqueos compartidos persistentes sobre las tuplas leídas, el sistema gestor permite que transacciones concurrentes modifiquen y confirmen cambios sobre dichas filas. Consecuentemente, las lecturas posteriores dentro de la frontera de la transacción original arrojarán valores dispares, generando un estado de inconsistencia temporal y comprometiendo críticamente la validez analítica y la exactitud de los reportes generados.

### Código de ejemplo:

Conexión A:

```
USE AdventureWorks2022;
GO
-- Nivel por defecto (previene lecturas sucias, pero permite
lecturas no repetibles)
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRAN;

-- Primera lectura: valor original
SELECT BusinessEntityID, SickLeaveHours AS Horas_Lectura_1
FROM HumanResources.Employee
WHERE BusinessEntityID = 2;

-- Tiempo para que la Conexión B modifique el dato
WAITFOR DELAY '00:00:10';

-- Segunda lectura: el valor ha cambiado dentro de la misma
transacción
SELECT BusinessEntityID, SickLeaveHours AS Horas_Lectura_2
FROM HumanResources.Employee
WHERE BusinessEntityID = 2;

COMMIT TRAN;
```

| Resultados |                  |                 |
|------------|------------------|-----------------|
|            | BusinessEntityID | Horas_Lectura_1 |
| 1          | 2                | 20              |
|            |                  |                 |
|            | BusinessEntityID | Horas_Lectura_2 |
| 1          | 2                | 100             |

Conexión B:

```
USE AdventureWorks2022;
GO
-- Se guarda el valor original en memoria para restaurarlo al final
DECLARE @OriginalHours SMALLINT = (SELECT SickLeaveHours FROM
HumanResources.Employee WHERE BusinessEntityID = 2);

-- Se modifica y confirma el cambio (cambio real temporal)
UPDATE HumanResources.Employee
SET SickLeaveHours = 100
WHERE BusinessEntityID = 2;

-- Se espera a que la Conexión A termine de leer este nuevo valor
WAITFOR DELAY '00:00:10';

-- Restauración la base de datos a su estado original
UPDATE HumanResources.Employee
SET SickLeaveHours = @OriginalHours
WHERE BusinessEntityID = 2;
```

---

(1 fila afectada)

(1 fila afectada)

Hora de finalización: 2026-09-22T06:12:39.0233693+02:00

## Conclusión:

La Conexión A obtuvo dos valores diferentes para el mismo registro dentro de una única transacción. El análisis demostró que el nivel READ COMMITTED protege contra datos no confirmados, pero no garantiza la estabilidad de los datos a lo largo del tiempo. Para prevenir este problema, se debe implementar el nivel de aislamiento REPEATABLE READ, el cual bloquea los datos recuperados hasta que la transacción finaliza, impidiendo actualizaciones por parte de otras conexiones.

### 3. Lecturas Fantasma (Phantom Reads)

#### Descripción del problema:

El problema de lecturas fantasma se suscita cuando una transacción consulta un subconjunto de filas delimitado por una condición de búsqueda (como una cláusula WHERE) y, durante el ciclo de vida de dicha transacción, un proceso concurrente ejecuta instrucciones de manipulación de datos que insertan o eliminan registros que satisfacen el mismo predicado. Al repetir la evaluación de la consulta original, el motor proyecta alteraciones en la cardinalidad del conjunto de resultados, introduciendo o retirando elementos (los denominados “fantasmas”). Este comportamiento vulnera severamente la consistencia de los modelos de agregación y los cálculos estadísticos consolidados.

#### Código de ejemplo:

Conexión A:

```
USE AdventureWorks2022;
GO
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRAN;

-- Se lee cuántos departamentos hay en el grupo 'Research and
Development'
SELECT COUNT(*) AS Total_Departamentos
FROM HumanResources.Department
WHERE GroupName = 'Research and Development';

-- Pausa para permitir la inserción fantasma
WAITFOR DELAY '00:00:10';

-- Se vuelve a contar: el número habrá incrementado (el fantasma)
SELECT COUNT(*) AS Total_Departamentos
FROM HumanResources.Department
WHERE GroupName = 'Research and Development';

COMMIT TRAN;
```

| Resultados Mensajes |   |
|---------------------|---|
| Total_Departamentos |   |
| 1                   | 3 |
|                     |   |
| Total_Departamentos |   |
| 1                   | 4 |

Conexión B:

```
USE AdventureWorks2022;
GO
BEGIN TRAN;
-- Se inserta un nuevo departamento que entra en la condición de la
Conexión A
INSERT INTO HumanResources.Department (Name, GroupName,
ModifiedDate)
VALUES ('Proyectos Especiales', 'Research and Development',
GETDATE());
COMMIT TRAN;

-- Espera a que la Conexión A termine su segunda lectura
WAITFOR DELAY '00:00:10';

-- Se elimina el registro fantasma para dejar la BD exactamente como
estaba
DELETE FROM HumanResources.Department
WHERE Name = 'Proyectos Especiales';

(1 fila afectada)

(1 fila afectada)

Hora de finalización: 2026-09-22T06:14:02.1912381+02:00
```

### Conclusión:

A pesar de utilizar REPEATABLE READ, la Conexión A reflejó un cambio en el conteo total. Esto se debe a que dicho nivel de aislamiento bloquea las filas existentes para evitar actualizaciones, pero no previene la adición de nuevas filas en el rango de búsqueda. Para solucionar este problema, es necesario elevar el nivel de aislamiento a SERIALIZABLE, el cual permite a SQL Server bloquear el rango de claves en el índice correspondiente, impidiendo que otras conexiones inserten o modifiquen datos cubiertos por la consulta.

## Conclusión general:

Durante la realización de la práctica se pudo observar la importancia que tienen los niveles de aislamiento al realizar transacciones en bases de datos que ya están en producción y por lo tanto presentan una alta concurrencia. De lo contrario, podrían existir problemas al acceder a datos erróneos, inexistentes o no considerados previamente, generando así problemas de consistencia e integridad en la base de datos.

Si bien, no hay un nivel de aislamiento que sirva para todas las situaciones, es importante tenerlos en cuenta al momento de programar una transacción.